

KavyaNest

E-Commerce Web Application Project Report

Developed by: **Kavya**

1. Introduction

1.1 Project Overview

KavyNest is a modern, responsive e-commerce web platform built with Python and the Django framework. It offers a diverse catalog featuring traditional ethnic wear—such as Kerala Kasavu sarees and Kurtha sets—alongside jewellery, footwear, beauty essentials, and electronics.

1.2 Purpose and Scope

The purpose of KavyNest is to bridge the gap between traditional retail and modern consumer expectations by providing a secure online shopping environment. Its scope covers the entire consumer journey: dynamic product catalogs, real-time search, persistent shopping carts, secure user authentication, and a streamlined checkout process.

1.3 Objectives

- Develop a scalable e-commerce architecture utilizing Django’s Model-View-Template (MVT) pattern.
- Create an intuitive, responsive UI using HTML5, CSS3, and Bootstrap 5.
- Establish secure authentication and robust relational database schemas.
- Deliver a complete, bug-free web application ready for deployment.

2. System Analysis

2.1 Existing vs. Proposed System

- **Existing System:** Traditional retail relies on physical presence, while many basic online catalogs lack interactive features, real-time inventory updates, or personalized experiences (like persistent carts and wishlists).
- **Proposed System (KavyNest):** Offers a centralized, bespoke Django application featuring real-time cart interactions, custom authentication (with security question password recovery), and a scalable backend designed for ease of use.

2.2 Requirement Specification

- **Hardware:** Client devices with standard web browsers. Server requiring minimum 2GB RAM and 20GB SSD space.
- **Software Frameworks:** Python 3.10+, Django 4.x / 5.x, SQLite3 (MySQL for Production), HTML5, CSS3, and Bootstrap 5.

3. System Design

3.1 Architecture Diagram

KavyNest adheres strictly to the **Model-View-Template (MVT)** architecture: * **Model:** Manages the SQLite database using Django's ORM. * **View:** Receives HTTP requests, executes business logic, and queries Models. * **Template:** Renders dynamic HTML using Django Template Language.

3.2 Data Flow & ER Modeling

- **Data Flow Diagram (DFD):** User input (search, auth, cart) flows through the View controllers, validating against the Database Models, and returns confirmation via rendered Templates.
- **Entity-Relationship (ER):** User has a one-to-one relationship with UserProfile and one-to-many relationships with CartItem, WishListItem, and Order. CartItem and WishListItem link back to Product.

3.3 UI/UX Wireframes

- **Home:** Fixed top navigation (Logo, Search, Cart). Hero banner above a dynamic product grid.
- **Cart/Checkout:** Tabular cart listing with quantity adjusters (+/-). Secure checkout form collecting shipping details and payment mode selection.

4. Implementation

4.1 Core Modules

1. **Authentication:** Secure login, registration, and custom “Forgot Password” flow utilizing user-defined security questions.
2. **Product Management:** Dynamic catalog rendering based on GET parameters for search queries, category filters, and price sorting.
3. **Shopping Cart:** Database-persistent cart linked to the user session, recalculating subtotals in real-time.
4. **Checkout:** Aggregates cart data, logs shipping details, simulates Cash on Delivery/Online Payment, and flushes the cart upon successful order creation.

4.2 Code Snippets

Database Model (store/models.py)

```
class Product(models.Model):
    name = models.CharField(max_length=200)
    category = models.CharField(max_length=50, choices=CATEGORY_CHOICES)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    image_url = models.CharField(max_length=500)

class Order(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    total_amount = models.DecimalField(max_digits=10, decimal_places=2)
    payment_mode = models.CharField(max_length=20)
    payment_status = models.CharField(max_length=20, default="Confirmed")
```

Order Processing View (store/views.py)

```
@login_required
def process_payment(request):
    mode = request.POST.get('payment_mode')
    cart_items = CartItem.objects.filter(user=request.user)

    total = sum(item.product.price * item.quantity for item in cart_items)
    status = "Paid" if mode == "ONLINE" else "Pending (COD)"

    order = Order.objects.create(user=request.user, total_amount=total,
                                payment_mode=mode, payment_status=status)
    cart_items.delete() # Flush cart
    return render(request, 'order_success.html', {'order': order})
```

5. Testing

5.1 Testing Strategies

- **Unit Testing:** Validating Product and Order model creations and required fields.
- **Integration Testing:** Ensuring the cart correctly creates database rows and increments quantities instead of creating duplicates.

5.2 Test Cases

ID	Scenario	Input	Expected Output	Status
TC-01	User Registration	Password: "abc", Confirm: "xyz"	Display error: "Passwords do not match."	Pass
TC-02	Product Search	Search Query: "Kasavu"	Display products containing "Kasavu"	Pass
TC-03	Cart Addition	Click "Add to Cart"	Cart item count increments by 1	Pass
TC-04	Order Checkout	Mode: COD, Cart has items	Order logged, Status: "Pending", Cart cleared	Pass

6. Conclusion and Future Work

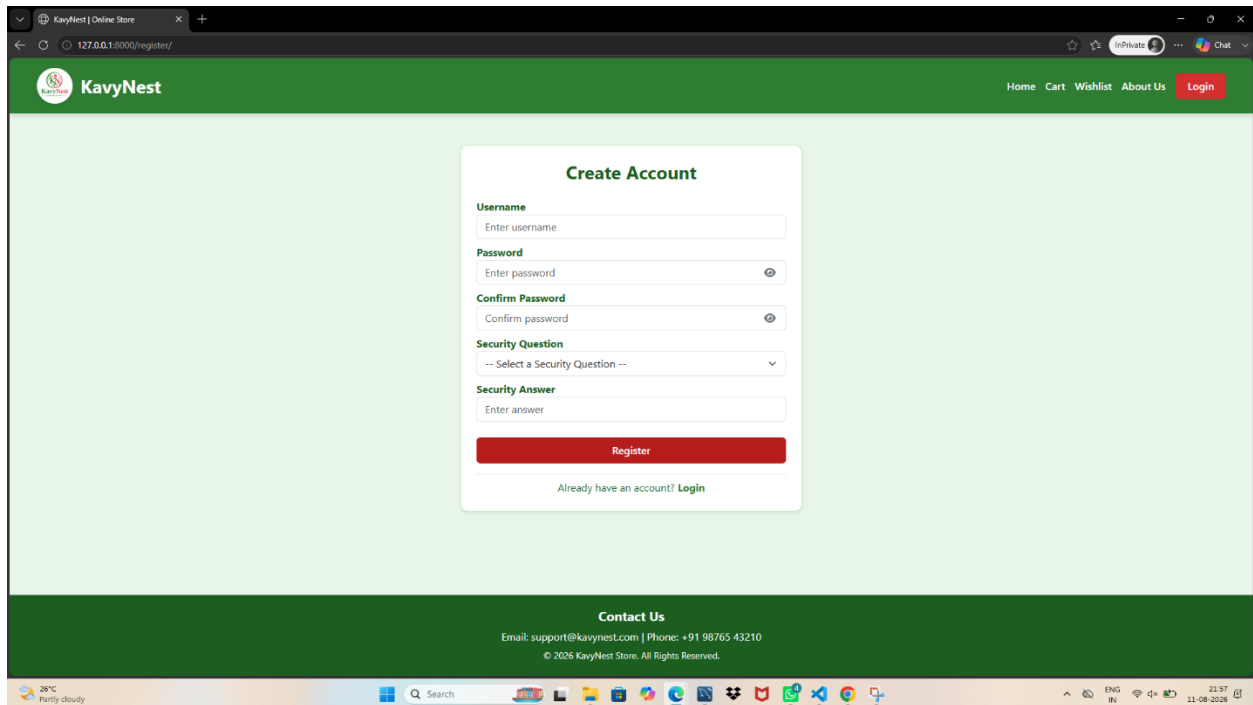
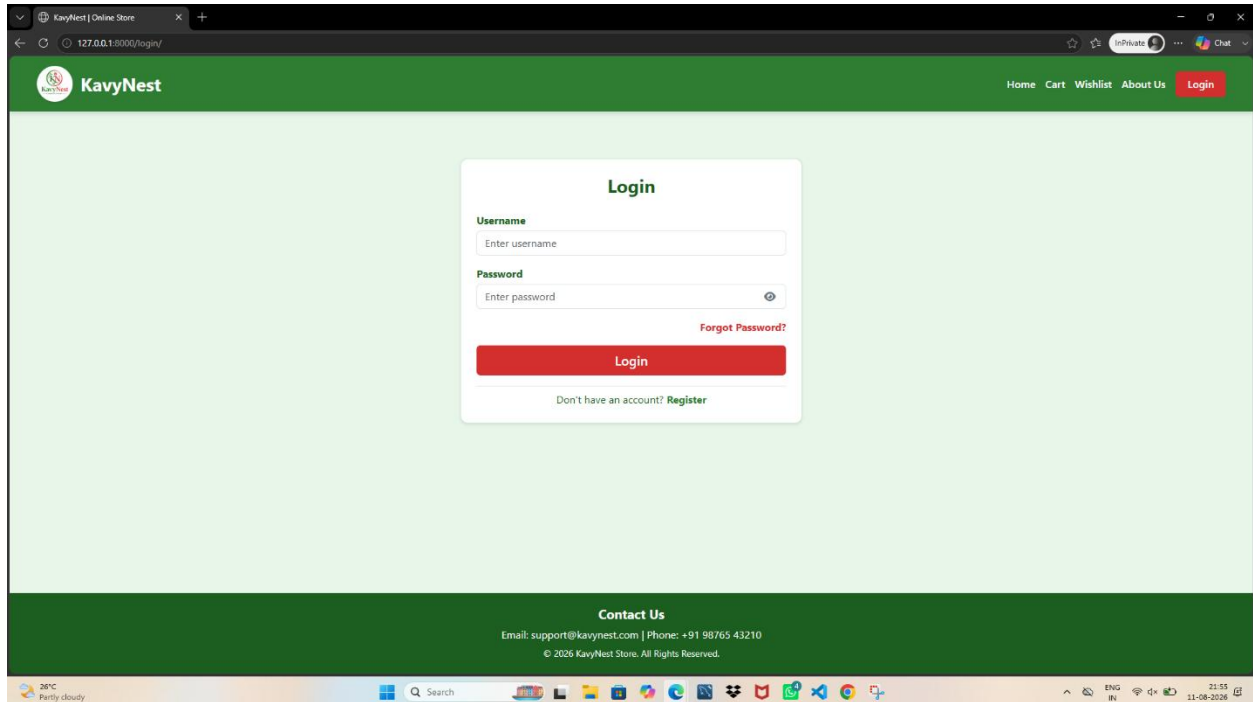
6.1 Conclusion

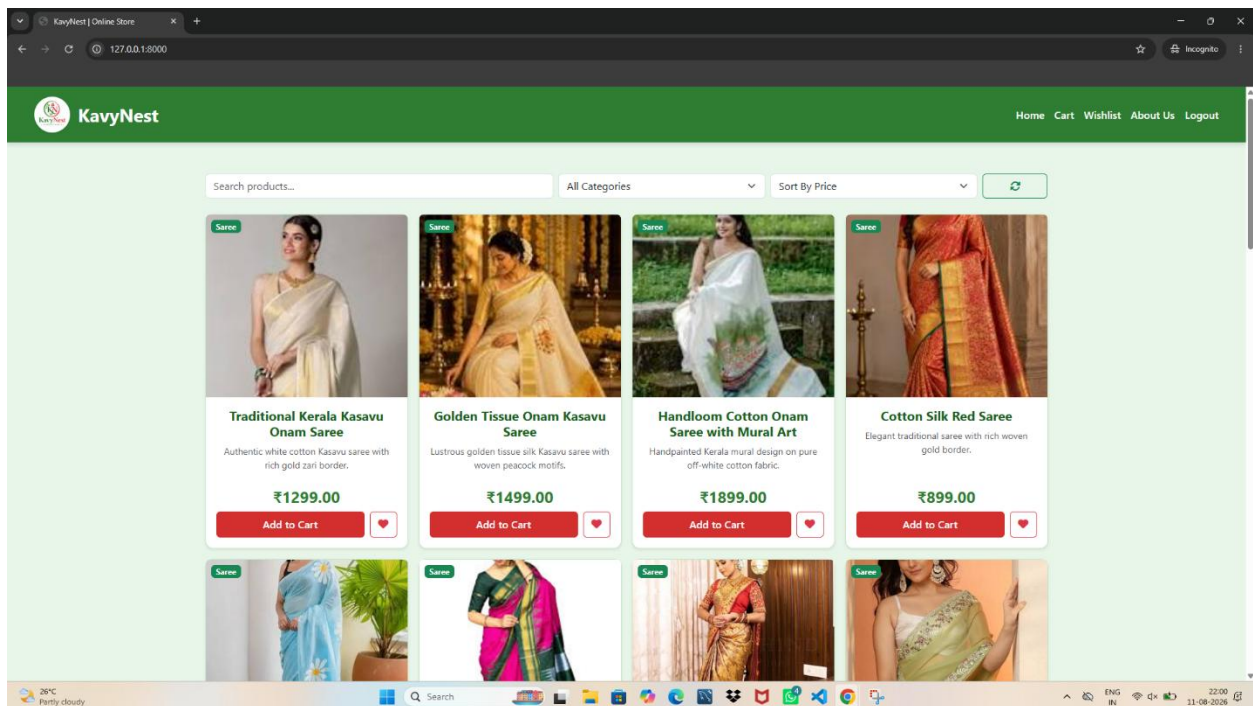
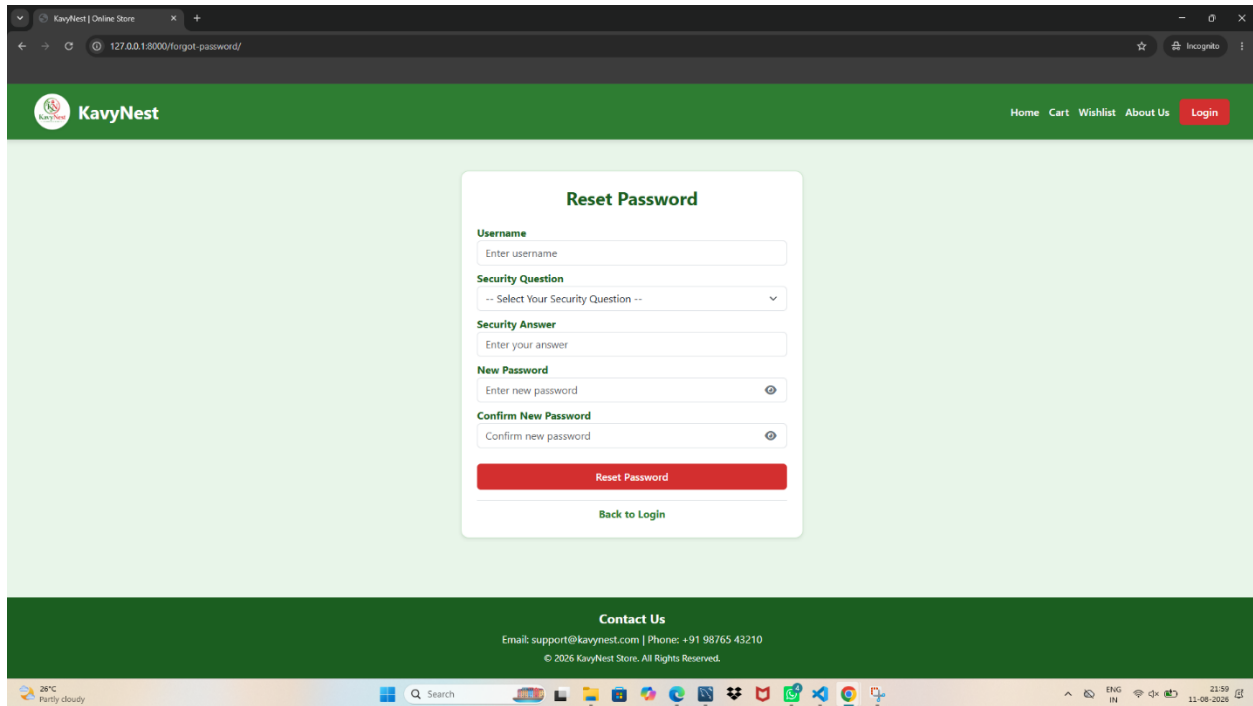
The KavyNest E-Commerce web application marks a significant achievement in building a comprehensive digital retail solution. By leveraging Python and Django, the project successfully implements a secure, scalable, and highly interactive platform that flawlessly met all initial objectives.

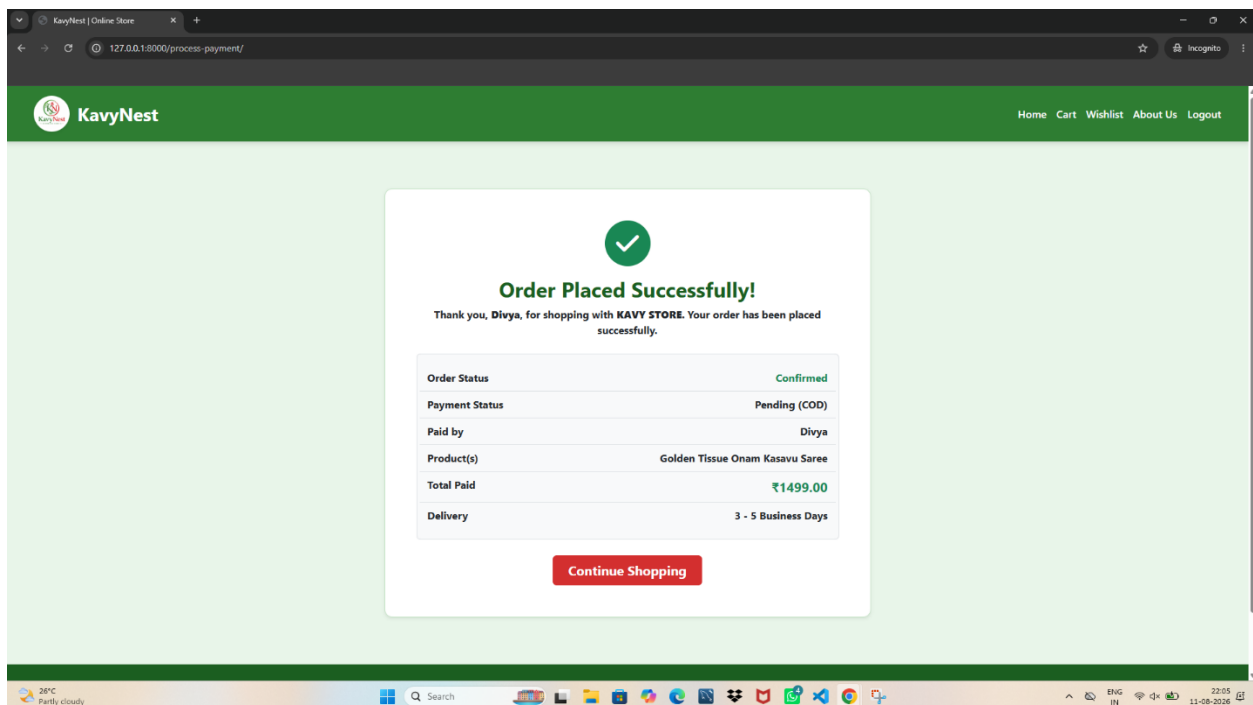
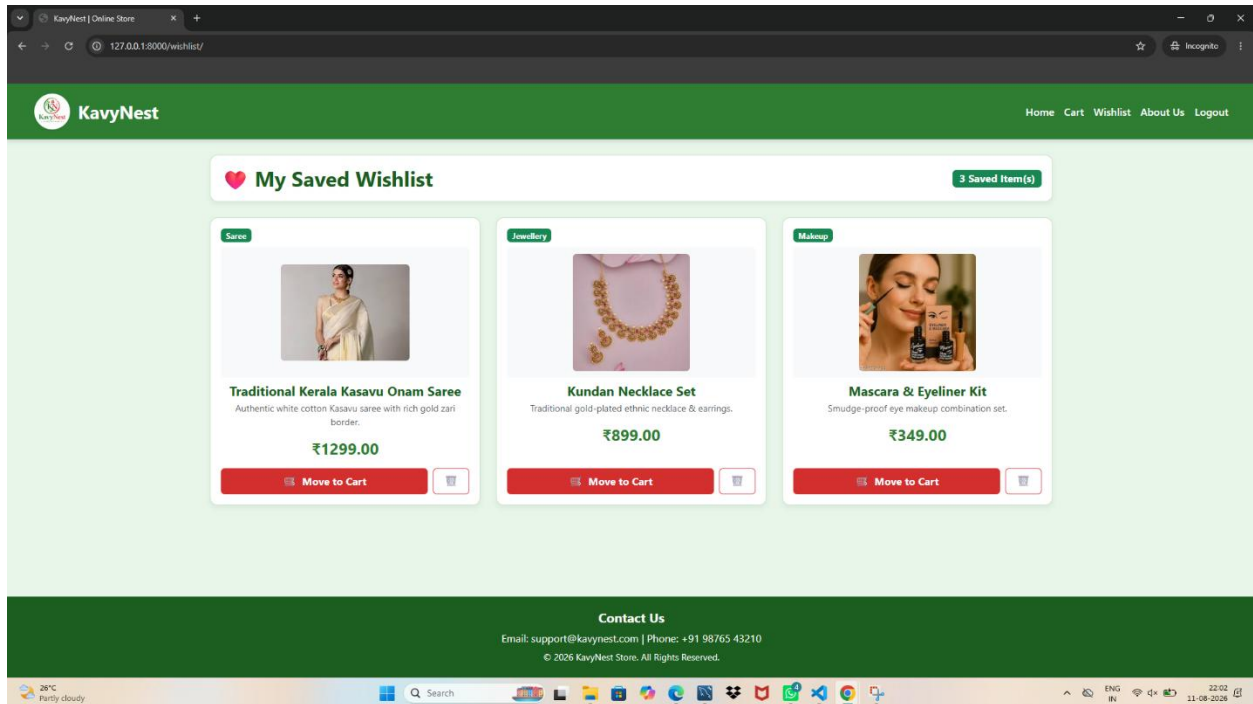
6.2 Future Enhancements

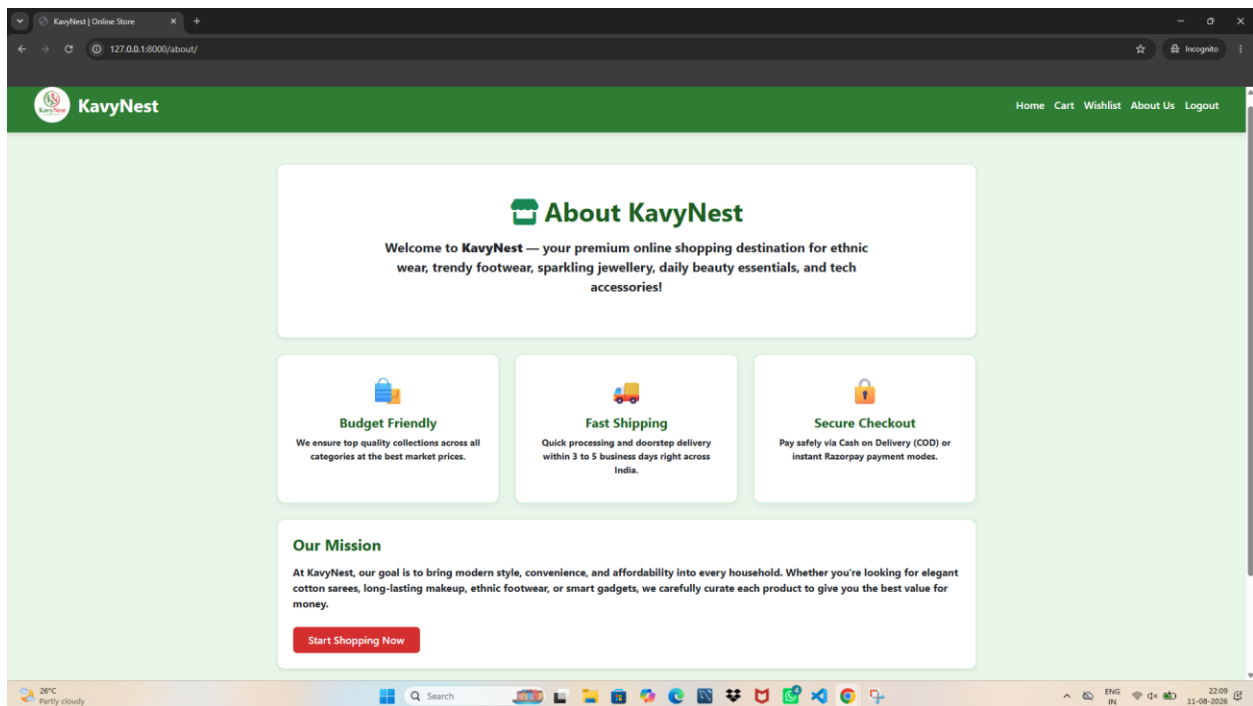
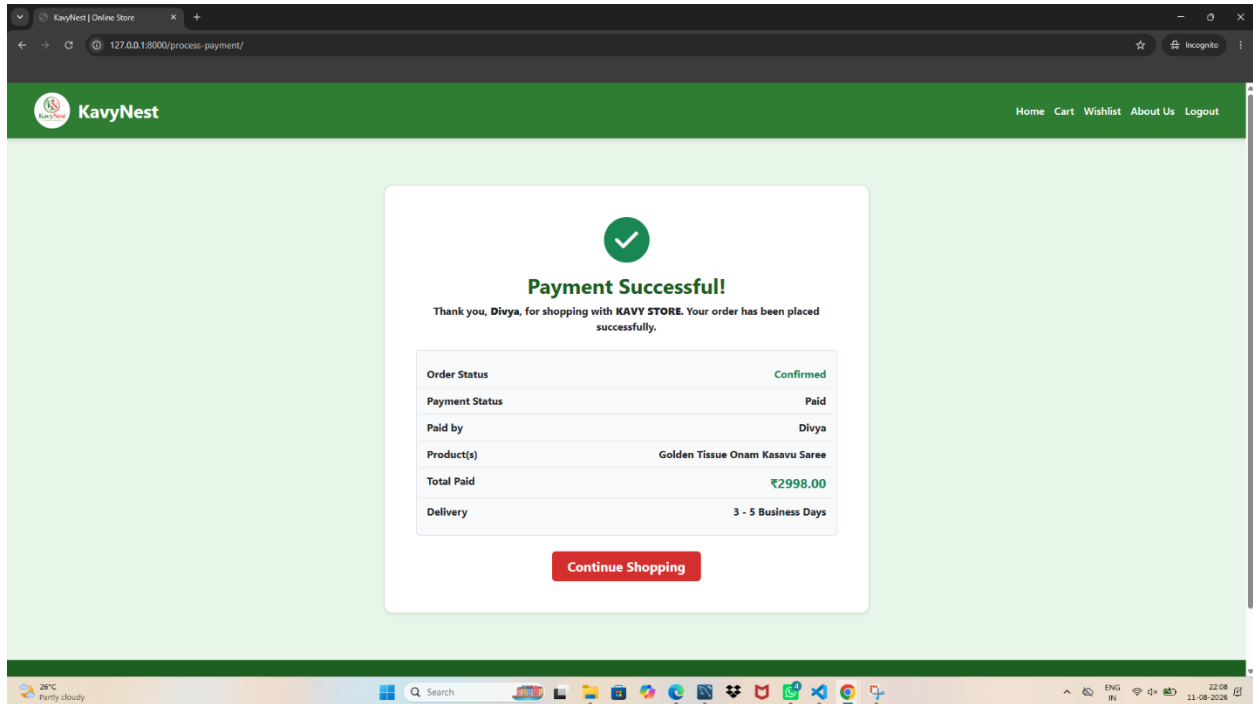
- **Payment Gateway Integration:** Integrating real APIs (Stripe, Razorpay).
- **Automated Emails:** Implementing SMTP for order confirmations and alerts.
- **Advanced Analytics:** Extending the admin panel with graphical sales charts.

6. Screenshots









The screenshot displays the MySQL Workbench environment. The top toolbar includes options for File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The left sidebar shows the 'SCHEMAS' list with 'kaynest' selected. The 'Tables' list under 'kaynest' includes tables like 'auth_group', 'auth_group_permissions', 'auth_permission', 'auth_user', 'auth_user_groups', 'auth_user_user_permissions', 'django_admin_log', 'django_content_type', 'django_migrations', 'django_session', 'store_cartitem', 'store_order', 'store_product', and 'store_userprofile'. The 'store_product' table is currently selected and displayed in the main area. The table has columns: id, name, price, description, image_url, and category. It contains 17 rows of product data. The bottom panel shows the 'Action Output' for a series of SQL queries executed, including 'SELECT * FROM kaynest.store_product' and 'SELECT * FROM kaynest.store_cartitem LIMIT 0, 1000'. The output shows the results of these queries, with the first query returning 17 rows and the subsequent queries returning 1 row each.

The screenshot displays the MySQL Workbench environment. The top toolbar includes icons for File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The left sidebar shows the 'SCHEMAS' tree with the 'kaynest' database selected. The main editor window contains the SQL query: `SELECT * FROM kaynest.store_order LIMIT 1000;`. Below the editor, the 'Result Grid' is visible, showing a table with 6 columns: `id`, `total_amount`, `payment_mode`, `payment_status`, `created_at`, and `user_id`. The first few rows of data are displayed. The bottom status bar shows the query execution progress, indicating it was completed successfully.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (dev... x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

acc
ecommerce_db
emp05
kaynest

Tables

auth_group
auth_group_permissions
auth_permission
auth_user
auth_user_group
auth_user_group_permissions
django_admin_log
django_content_type
django_migrations
django_session
store_cartitem
store_order
store_product
store_userprofile

Columns
Indexes
Foreign Keys
Triggers
store_wishlistitem

Administration Schemas

Information

Schema: kaynest

1 SELECT * FROM kaynest.django_migrations;

Result Grid

#	id	app	name	applied
1	0001	contenttypes	0001_initial	2026-08-11 14:06:30.041998
2	0001	auth	0001_initial	2026-08-11 14:06:31.459463
3	0001	admin	0001_initial	2026-08-11 14:06:31.712757
4	0002	auth	0002_alter_user_first_name_max_length	2026-08-11 14:06:31.731124
5	0003	auth	0003_alter_user_email_max_length	2026-08-11 14:06:31.746746
6	0004	auth	0004_alter_user_username_max_length	2026-08-11 14:06:31.793238
7	0005	auth	0005_alter_user_last_login_null	2026-08-11 14:06:32.119455
8	0006	auth	0006_require_contenttypes_0002	2026-08-11 14:06:32.179418
9	0007	auth	0007_alter_permission_name_max_length	2026-08-11 14:06:32.305410
10	0008	auth	0008_alter_group_name_max_length	2026-08-11 14:06:32.336152
11	0009	auth	0009_alter_user_last_name_max_length	2026-08-11 14:06:32.322151
12	0010	auth	0010_alter_group_name_max_length	2026-08-11 14:06:32.394054
13	0011	auth	0011_update_permissions	2026-08-11 14:06:32.483514
14	0012	auth	0012_alter_user_first_name_max_length	2026-08-11 14:06:32.639401
15	0013	auth	0013_alter_group_name_max_length	2026-08-11 14:06:32.681526
16	0014	auth	0014_alter_group_name_max_length	2026-08-11 14:06:32.698783
17	0015	auth	0015_alter_group_name_max_length	2026-08-11 14:06:32.698783

Output

#	Time	Action	Message	Duration / Fetch
19	21:47:01	SELECT * FROM kaynest.store_product LIMIT 0, 1000	21 row(s) returned	0.000 sec / 0.000 sec
20	21:47:05	SELECT * FROM kaynest.store_wishlistitem LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
21	21:47:08	SELECT * FROM kaynest.store_cartitem LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
22	23:19:15	SELECT * FROM kaynest.store_cartitem LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
23	23:19:16	SELECT * FROM kaynest.django_session LIMIT 0, 1000	1 row(s) returned	0.015 sec / 0.000 sec
24	23:19:19	SELECT * FROM kaynest.django_migrations LIMIT 0, 1000	20 row(s) returned	0.015 sec / 0.000 sec

Object Info Session

25°C
Partly cloudy

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

MySQL Workbench

Local instance MySQL80 x Local instance MySQL80 (dev... x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

acc
ecommerce_db
emp05
kaynest

Tables

auth_group
auth_group_permissions
auth_permission
auth_user
auth_user_group
auth_user_group_permissions
django_admin_log
django_content_type
django_migrations
django_session
store_cartitem
store_order
store_product
store_userprofile

Columns
Indexes
Foreign Keys
Triggers
store_wishlistitem

Administration Schemas

Information

Schema: kaynest

1 SELECT * FROM kaynest.store_wishlistitem;

Result Grid

#	id	product_id	user_id
8	1	7	7
9	21	7	7
10	15	7	7

Output

#	Time	Action	Message	Duration / Fetch
24	23:19:19	SELECT * FROM kaynest.django_migrations LIMIT 0, 1000	20 row(s) returned	0.015 sec / 0.000 sec
25	23:20:12	SELECT * FROM kaynest.store_cartitem LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
26	23:20:14	SELECT * FROM kaynest.store_order LIMIT 0, 1000	8 row(s) returned	0.000 sec / 0.000 sec
27	23:20:33	SELECT * FROM kaynest.store_product LIMIT 0, 1000	21 row(s) returned	0.015 sec / 0.000 sec
28	23:20:53	SELECT * FROM kaynest.store_userprofile LIMIT 0, 1000	7 row(s) returned	0.016 sec / 0.000 sec
29	23:20:59	SELECT * FROM kaynest.store_wishlistitem LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec

Object Info Session

25°C
Partly cloudy

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.